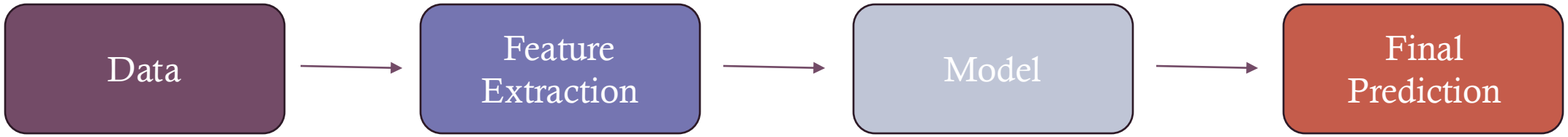
MACHINE LEARNING AND DEEP LEARNING

Lab 02: Image Classification

Dr. Giacomo Lucato, a.y. 2025-2026

CLASSIFICATION – MACHINE LEARNING PIPELINE

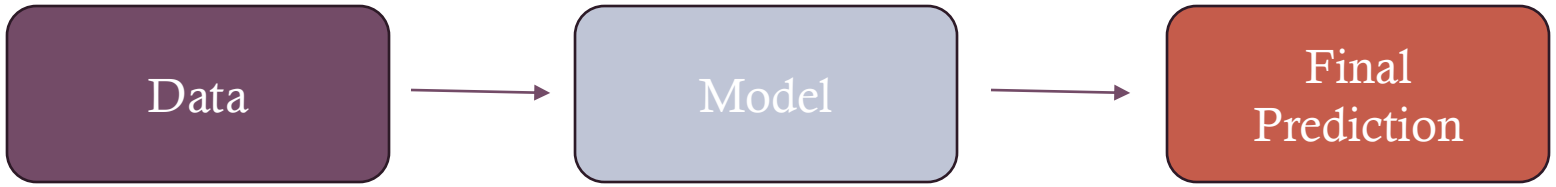
- Classification is a ML task involving assigning each data sample a **discrete label,** from a predefined, fixed number of classes.
- For standard Machine Learning, the general pipeline for classification can be represented through the following schema:



- Notice that the Feature Extraction phase is **explicit.**

CLASSIFICATION – DEEP LEARNING PIPELINE

- In Deep Learning, the classification pipeline is similar to ML, yet there is a fundamental difference: the Feature Extraction phase is **no longer explicit.**
- Instead, **features** are **automatically learned** and extracted directly by the DL model.

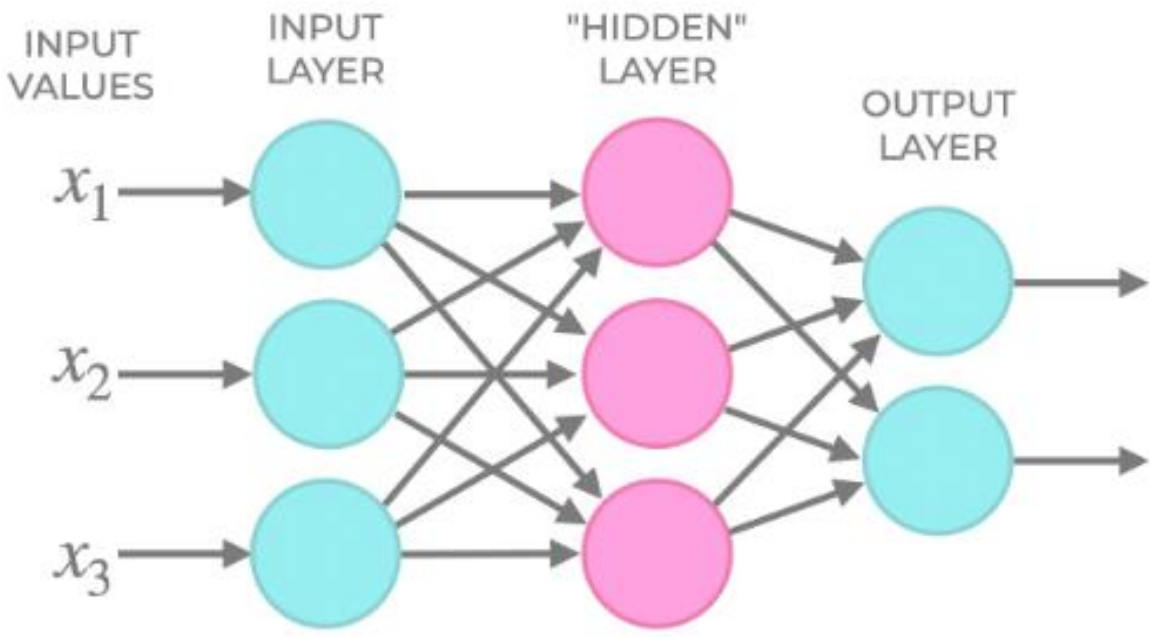


- This difference is one of the main distinctions between ML and DL.

MULTI-LAYER PERCEPTRON (MLP)

- The first neural networks one can think of to solve a classification task is the **Multi-Layer Perceptron (MLP).**
- A Multi-Layer Perceptron (MLP) is a **feed-forward neural network** consisting of at least three layers of nodes (input, hidden, and output), where every neuron in one layer is **connected to every neuron** in the next.
- It uses **non-linear activation functions** (like ReLU) to learn complex mappings between raw input data and target labels.

MULTI-LAYER PERCEPTRON (MLP)



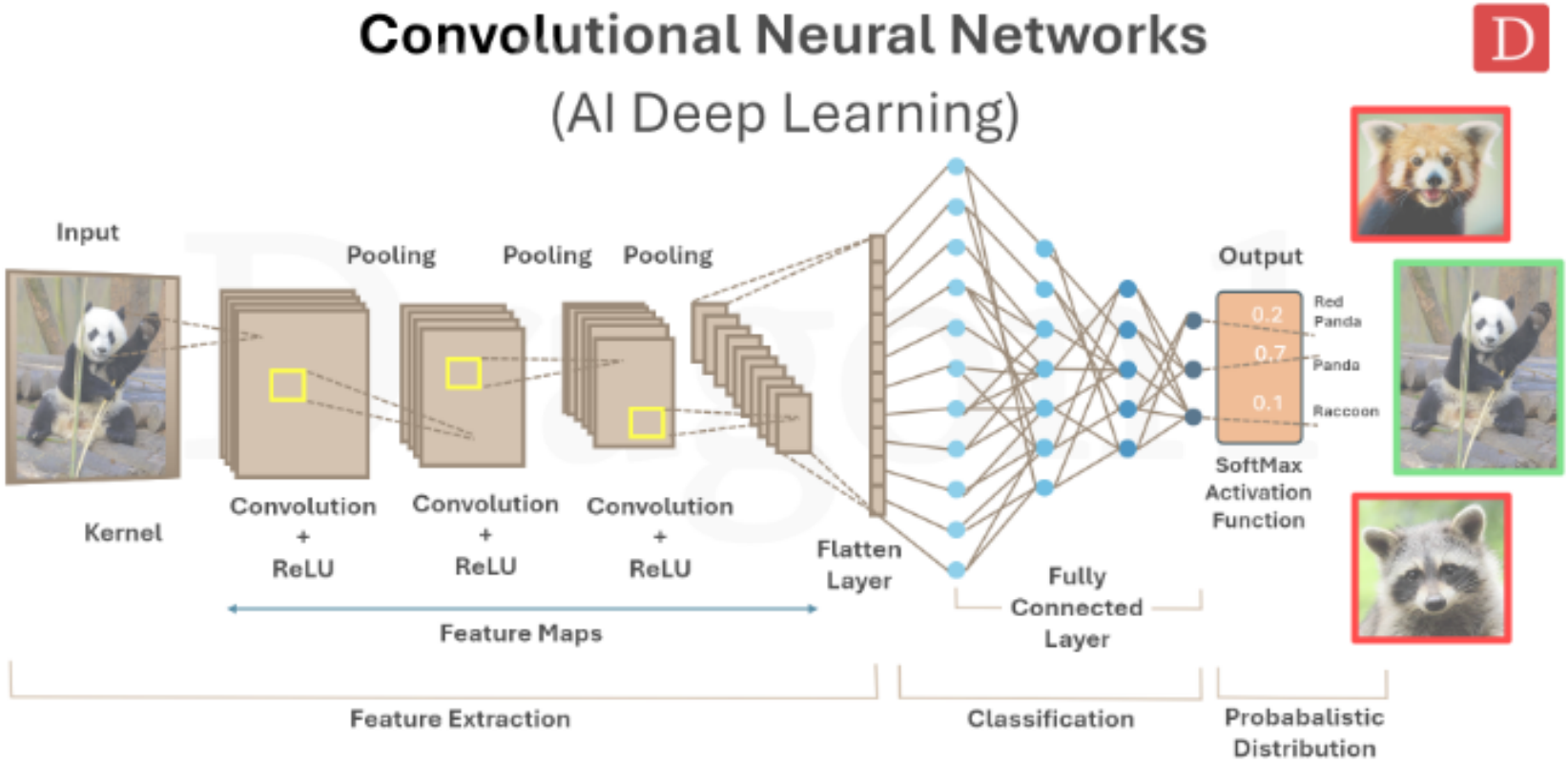
MULTI-LAYER PERCEPTRON (MLP)

- MLPs are generally unsuitable for Image-based tasks, for three main reasons:
  - **Loss of Spatial Topology:** in an image, pixels have a meaning only considering their neighbors. However, when feeding an image into an MLP, it must be flattened in a 1d-vector, losing positional information of single pixels.
  - **Lack of Translation Invariance:** if important details in an image are translated even by few pixels, this would represent a completely new input for an MLP. Therefore, MLPs are not able to recognize the same shapes at different positions across images.
  - **Curse of Dimensionality:** in MLPs, the number of parameters explodes exponentially even at the first layer, making the model slow, hard to train and extremely prone to overfitting.
- E.g., a first layer containing 1024 neurons, accepting 224x224 images in RGB, would have approximately over 154 millions parameters to train.

CONVOLUTIONAL NEURAL NETWORKS (CNN)

- A Convolutional Neural Network (CNN) is a **deep learning architecture** designed to process data with a **grid-like topology**, such as images.
- It is composed of three main types of layers:
  - **Convolutional Layers:** these layers act as filters, extracting hierarchical features through convolutions with the input images. These are learnable filters.
  - **Max Pooling Layers:** these layers reduce the dimensionality of the data, by usually picking the highest value from a 2x2 square of pixels and discarding other values.
  - **Fully Connected Layers:** these layers are placed at the end of the network, and they are responsible to make the final prediction based on the features they received from previous layers.

CONVOLUTIONAL NEURAL NETWORKS (CNN)



TRAINING A DEEP LEARNING MODEL

- Training a Deep Neural Network involves two main steps: the **Forward pass** and the **Backward pass.**
- The **Forward pass** involves computing the **predictions** of the model for some data samples.
- Once we have the predictions, we **compare them with the ground truth,** and we compute the **loss** according to some criteria, depending on the current task.
- The objective of the **Backward pass** is to adjust the parameters (weights) of the network to **minimize this loss.**
- This minimization is typically carried on through a **Gradient Descent update** of the weights of the network (Chain Rule).

CROSS ENTROPY LOSS

- For classification tasks, the loss function mostly used is the **Cross Entropy Loss.**

$$\text{Loss} = -\sum_{i=1}^{C} y_i \cdot \log(\hat{y}_i)$$

- It is computed as the **sum over all the classes** of the **ground truth** (yi) times the natural logarithm of the **model's prediction** (which is a probability for the class i).
- The idea is that if the model is confident about a wrong prediction, it will get a high loss; on the other hand, if it is confident about a right prediction, the loss will be low.

EXERCISE

- **Assignment:** follow the example *cards_classification.ipynb* to write your image classification task.
- Load the digits.zip dataset, unzip it, and visualize 2 images for each class.
- Assess the data distribution across different classes.
- Write a transformation pipeline (resizing, normalization and augmentation) for training and test data, and apply it creating two ImageFolder objects.
- Create train_dataloader and test_dataloader.
- Define an MLP and a CNN (with and without Dropout) for the task.
- Define your training loop, defining optimizers and loss function.
- Train your models.
- Compare your models' performance on the test set.